

CPT304 Software Engineering 2 - Tutorial 3

Theory

Question 1

Let's assume you have an application where you need to write logs to a file. This could be anything from info logs, debug logs to error logs.

The following needs are to be addressed here:

1. All parts of an application need to write to the same log file.
2. Only one part of the application should be able to write to the log file at a time to avoid concurrency issues.

Which design pattern should you use as the solution for the problem?

Answer: This is an ideal situation for the Singleton pattern because:

1. Every part of the application can access the singleton logging class since it's globally accessible.
2. Since there is only one instance of the logging class, you avoid multiple parts of the application from writing to the file simultaneously.

Question 2

Imagine a scenario where you are developing a video streaming service, similar to Netflix or YouTube.

Streaming a video is a resource-intensive operation, especially for high-definition videos. If your service were to directly stream videos from their original location for every single user request, it could lead to a lot of latency, high bandwidth usage, and potentially overloading the server.

Instead of directly streaming the video from its original location, you initially only holds the reference to the original video. When the user clicks on a video to watch, you will then stream the video from the original location, buffers it, and sends the buffered data to the client. Which design pattern is suitable for the problem?

Answer: Proxy pattern can be an excellent solution. Instead of directly streaming the video from its original location, you can create a VideoProxy which holds the reference to the original video. When the user clicks on a video to watch, the proxy then streams the video from the original location.

Question 3

Consider a scenario where you're developing an application for a Vehicle Rental Service. This service can rent out several types of vehicles like cars, trucks, and motorcycles.

Customers can select the type of vehicle they want to rent. Based on the customer's choice, the system must create an object of that type. If you were to create these objects directly within your application, it could lead to a lot of repetitive, error-prone code, and it would make your code harder to maintain and extend. Which design pattern is most suited to this scenario?

Answer: This is where the Factory pattern shines. You can create a VehicleFactory that has a method createVehicle(String type). This method takes the type of vehicle as an argument and returns an object of that type.



Question 4

Consider a scenario in which you're designing a data analytics tool that uses data from multiple sources. These sources could include different databases (SQL, NoSQL), CSV files, and even APIs that return JSON.

Each of these data sources has a different way of delivering the data. SQL databases use SQL queries, NoSQL databases have their own query syntax, CSV files need a file reader and JSON APIs require HTTP requests. Moreover, the returned data from each source is in a different format.

If you were to integrate every data source directly into your system, it would result in a lot of redundant and hard-to-maintain code, as you would have to handle each data source and format separately. Which design pattern is suitable for the problem?

Answer: This is where the Adapter pattern comes into play. For each data source, you could create an adapter that translates the particular data source to a common interface that your system understands.

Question 5

Consider a file system. In a file system, there are two types of entities: File and Directory.

A File is a leaf node that has content and size, while a Directory can contain multiple files or directories but doesn't have content of its own, its size is typically the total of sizes of all files it contains.

To develop the file system described above, which design pattern is most suitable for the problem?

Answer: This is a perfect use-case for the Composite design pattern, as it allows clients to treat individual File objects and compositions of File and Directory objects uniformly.



Question 6

Consider a scenario where a text-editing application that allows users to apply different types of formatting to the text.

Let's say that you have plain text and you want to allow users to apply bold, italic, underline, or strikethrough formatting. The user should be able to apply these formats independently or in combination. Which design pattern is the most suited solution for the problem?

Answer: Decorator design pattern can perfectly resolve the problem. You could program BoldText, ItalicText, UnderlineText, and StrikethroughText classes as decorators which, act as a wrapper to the original context represented in PlainText class.

装饰器：动态给对象追加功能多层嵌套。